

AIJOB-BASELINE.md

Fonte di verità per AiJob. Non la chat.

Baseline corrente

Campo	Valore
Artefatto	index.html (AiJob v0.20.0)
SHA-256	453b07d90f85a005f4bd2a616e1aff3ac613a2e86c6cd5fd63a70e1f6ee57120
Righe	2834
Validazione	node --check OK
Test	154/154 jsdom
Data	9 agosto 2026
Ratifica umana	in attesa

Come verificare (non fidarti della dichiarazione)

```
sha256sum index.html
npm install jsdom
node smoke8.mjs
node contrast.mjs # audit contrasto WCAG, deve dare 0 fallimenti
```

smoke.mjs è allegato. Se un agente vuole contestare "9/9 dichiarato da Claude", esegue il file e ottiene il proprio risultato.

Storico

Versione	SHA-256 (primi 12)	Cambiamento
0.1.0	00cafd8471c9	Prima versione funzionante

Versione	SHA-256 (primi 12)	Cambiamento
0.1.1	fbff2972ac61	Correzione difetto bloccante + robustezza errori
0.1.2	72613c55bdd3	Data errata corretta, verifica modelli via API, guardia prompt injection
0.1.3	9794442b11eb	Conformita WCAG AA: contrasto, tipografia, stati non solo a colore
0.2.0	a0c49938ab8b	AiBadge, doppio punteggio, classifica, progressive disclosure
0.3.0	4c02583d5353	AiSearch assistita, selezione multipla, dossier in blocco, invio multicanale
0.4.0	ca0390fa173b	Fusione col Registro candidature; vincolo postazione; CSV; filtri fase
0.5.0	87292214e6f0	Google collegato: PDF veri, bozza Gmail con allegati, Drive, Calendar
0.6.0	15b4260a391d	Diagnostica Google; analisi dei gap ricorrenti e formazione
0.7.0	d5d3ed219d64	OAuth per reindirizzamento, zero librerie esterne
0.7.1	cc24fc418556	Stadio di fallimento esplicito; rilevamento anteprima; avviso permanente
0.7.2	735b2c628008	Rilevamento Brave corretto; collegamento manuale; fallback in memoria
0.8.0	16095259e51c	Ricerca reale in rete; diagnosi del 403 per API e permessi
0.9.0	0d3f394b29ba	Punteggio opportunità, freschezza, rischio annuncio fantasma, budget ricerca, backup Drive
0.9.1	a29dbe1989d1	Motore di ricerca multi-provider con ricaduta automatica
0.9.2	5151246dbc87	Prova ATS deterministica; scheda Info con mappa AiWorld
0.10.0	12fecef04b2b	Scheda Oggi come homepage; stato condiviso AiWorld su Drive

Versione	SHA-256 (primi 12)	Cambiamento
0.11.0	9a2a605be4c4	Canale di candidatura e Kit portale
0.12.0	2b6a44fa1f36	Tavily: ricerca separata dal ragionamento; registro agenti
0.20.0	453b07d90f85	Merge chirurgico su 0.19.1: AiSetup, AiAgents, rete di sicurezza
0.19.1	524aeab5914a	Marchio originale AiWorld e icona del sito
0.19.0	7a692aa8f44a	Invio reale, diagnostica che prova le API, distanza, video
0.18.2	157638bfd9bd	Radar in un file solo: zero chiavi, zero segreti, zero configurazione
0.18.1	d056aa0f3368	Radar v4: API pubbliche di jobs.ch e jobup.ch
0.18.0	0478742fd39b	Radar v3: lettura dei dati strutturati JobPosting dai portali
0.17.1	d8014ab7274e	Radar v2: adattatori, feed RSS senza chiavi, Adzuna facoltativo
0.17.0	3a9a8208059a	Radar: raccolta annunci lato server via GitHub Actions
0.16.0	fe592da5dbdb	Adzuna: annunci strutturati veri; analisi in blocco
0.15.0	6ee55e6cb814	Autodiagnosi e autocorrezione; scelta dei modelli gratuiti
0.14.0	e271264f1ff0	Sfondo piu' profondo; diagnostica HTTP per codice; OpenAI dichiarato non chiamabile
0.13.1	05e1f6cac202	Correzione di due regressioni: gestori perduti e contatore bloccante
0.13.0	e72b4361d26e	Registro provider: chiave e basta, catena con ricaduta automatica

0.1.1 — dettaglio

Difetto bloccante: il modello di default era `gemini-2.0-flash`, spento da Google il 1 giugno 2026. L'app avrebbe fallito alla prima chiamata.

- Default → `gemini-3.6-flash`; elenco di modelli verificati nel campo
- Rimosso `temperature` dalla chiamata Gemini: deprecato sui modelli 3.x
- `maxOutputTokens` 4096 → 8192: CV + lettera + email eccedevano il limite e il JSON veniva troncato
- Errori 404 e 429 con messaggio comprensibile invece del corpo grezzo della risposta

Nessuna feature aggiunta. Nessuna modifica al dominio.

0.1.2 — dettaglio

Verifica alla fonte primaria (release notes ufficiali Gemini API, ultimo aggiornamento 30 luglio 2026), non su memoria né su dichiarazioni di agenti:

- `gemini-3.6-flash` **esiste**, GA dal 21 luglio 2026. Il dubbio sollevato da GPT è respinto con evidenza.
- `gemini-2.0-flash` spento il **1 giugno 2026**. La mia dichiarazione "31 marzo" in 0.1.1 era **errata**: corretta nel codice e qui.
- `temperature`, `top_p`, `top_k` deprecati dal 21 luglio 2026. La loro rimozione in 0.1.1 era corretta, per la ragione giusta.

Correzioni applicate:

- Pulsante **Elenca modelli**: interroga `GET /v1beta/models` con la chiave dell'utente e popola il campo con i modelli realmente disponibili. Elimina in modo permanente la classe di difetto "nome modello presunto" — nessun agente deve più fidarsi di una dichiarazione.
- Guardia prompt injection: il testo dell'annuncio è delimitato ed etichettato come dato non fidato; le regole vengono ripetute dopo il blocco; le istruzioni contenute nell'annuncio vanno registrate in `red_flags` invece che eseguite.

D-06 chiuso. D-05 invariato.

Ambito congelato

Fino a validazione su 3 annunci reali:

- Nessuna feature nuova. Solo correzione di difetti emersi dall'uso.
- Correzioni ai prompt: consentite, sono configurazione, non architettura.
- `AiWork`, `AiWorld`, adapter Google: fuori ambito.

Difetti noti, dichiarati

#	Difetto	Gravità
D-01	Chiave API in chiaro in <code>localStorage</code>	● accettabile su telefono personale, non su dispositivo condiviso
D-02	Nessuna lettura del link: CORS e anti-bot bloccano il <code>fetch</code> da browser	● per progetto, fallback = incolla
D-03	<code>localStorage</code> cancellabile dal browser senza preavviso	● mitigato da esporta JSON
D-04	Nessun conteggio token: un annuncio molto lungo può ancora troncarsi	● da osservare nell'uso
D-05	Il Claim Firewall è nel prompt, non nel codice: non è verificabile a runtime	● è il difetto architetturale vero . Se il modello ignora la regola, nulla lo blocca

| D-06 | Prompt injection dall'annuncio: un testo ostile poteva istruire il modello a dichiarare requisiti soddisfatti | ● **mitigato in 0.1.2** — mitigazione nel prompt, non dimostrata |

| D-07 | 5 coppie di colore sotto la soglia WCAG AA, incluse le etichette del registro evidenze | ● **risolto in 0.1.3** — verificabile con `contrast.mjs` |

| D-08 | Lo stato dell'evidenza era comunicato dal solo colore (viola WCAG 1.4.1) | ● **risolto in 0.1.3** |

D-05 è la ragione per cui la validazione su annunci reali non è una formalità.

D-06 è mitigato ma non risolto: la difesa vive nello stesso prompt che dovrebbe proteggere.

Rapporto con gli altri prodotti

AiJob non condivide repository, baseline né stato con AiEvents. Prodotti distinti.

`AiBridge` in questo progetto è il tool clipboard v0.2.1 — non l'integration layer descritto nel briefing AiWork. La collisione di nome non è risolta.

0.1.3 — dettaglio

Audit di leggibilità e accessibilità, misurato non stimato. `contrast.mjs` calcola i rapporti di contrasto secondo la formula WCAG e va eseguito a ogni modifica della palette.

Prima: 5 fallimenti su 12 coppie. Le tre etichette del registro evidenze — l'elemento che distingue AiJob da ogni competitor — erano tutte sotto soglia: SUPPORTED 3.20:1, NOT_SUPPORTED 3.61:1, contro un minimo di 4.5:1.

Dopo: 0 fallimenti su 12. Stesse tinte, luminosità corretta.

Token	Prima	Dopo	Su card
--ok	#2f7d67 (3.20)	#38997d	4.53
--signal	#d64545 (3.61)	#dd6464	4.58
--warn	#b8862b (4.88)	#b2842a	4.69
--line	#39424f (1.79)	#57667a	3.10

Altre correzioni:

- Nessun testo sotto i 12px. Le etichette di stato passano da 10px a 12px in semigrassetto.
- Tab e pulsanti di stato: area di tocco minima 40–48px, testo da 11px a 14px, niente maiuscolo forzato.
- Interlinea 1.5 → 1.6; documenti generati a 16px con riga limitata a 66 caratteri.
- **Lo stato non dipende più dal colore:** ✓ CON PROVA , ≈ PARZIALE , ✗ SENZA PROVA , ? DA VERIFICARE . Leggibile in scala di grigi, da daltonici e da uno screen reader.
- `aria-live` sui messaggi di stato, `role="alert"` sui toast.
- `prefers-reduced-motion` rispettato su transizioni, animazioni e scroll.

Nota di metodo. Due sostituzioni della patch erano fallite in silenzio: una stringa ancora era già stata modificata da una sostituzione precedente. Se ne sono accorti i test,

non io. Da qui in avanti ogni sostituzione va con `assert` sull'ancora.

0.2.0 — dettaglio

Primo cambio di versione minore. Rottura consapevole del freeze concordato dagli agenti,

su decisione di Romeo, con questa motivazione: il gate dei 3 annunci reali non veniva raggiunto perché l'attrito iniziale era troppo alto. La v0.2 riduce quell'attrito.

Chiave di storage nuova (`aijob.v2`): la v0.1 non viene migrata.

AiBadge. Il CV si incolla una volta. Il modello ne estrae fatti atomici e li marca `DOCUMENTO` (una frase del CV lo sostiene alla lettera) o `DEDOTTO` . I dedotti stanno in cima e vanno confermati o eliminati a mano; diventano `CONFERMATO` . **Solo i fatti non dedotti vengono passati al modello** in analisi e generazione: la deduzione non può diventare silenziosamente un fatto.

Due punteggi, mai mediati.

	Domanda	Dimensioni
So farlo	La persona e' in grado?	competenze, esperienza, lingua, seniority, formazione
Mi conviene	Vale il tempo speso?	luogo, salario, contratto, crescita, facilita candidatura

Divergono spesso, ed e' il punto: fit 88 / convenienza 52 e' un annuncio da lasciar perdere
malgrado il fit. Nessun competitor separa le due domande. L'obiettivo dichiarato (assunzione, salario, vicinanza, crescita, velocita) pesa sul secondo punteggio.

Classifica. Ordinata sulla somma dei due. Ogni scheda mostra solo posizione, azienda, ruolo, due numeri e un verdetto. Il resto e' dietro "Perche'?" — progressive disclosure con `aria-expanded` / `aria-controls`. Nessun decimale di confidenza esposto: bande qualitative.

Gap colmabili. Per ogni requisito mancante il modello propone azione e tempo stimato.

Non implementato (radar): scoperta automatica, adapter Google, AiApply, AiCV, Claim Firewall deterministico, agents.json.

Nota sulla scoperta automatica

L'API pubblica ufficiale svizzera (`api.job-room.ch/jobAdvertisements/v1`) serve ai **datori di lavoro per pubblicare** annunci, non ai candidati per cercarli. Non esiste quindi, a oggi, una fonte gratuita e legale di discovery automatica per il caso Ticino. L'ingresso resta manuale. Verificato alla fonte, 9 agosto 2026.

0.3.0 — dettaglio

Richiesta di Romeo: selezionare gli annunci dalla classifica e avviare una seconda procedura di candidatura; sostituire la classifica con AiSearch.

Cosa e' stato costruito

- Casella di selezione su ogni annuncio; barra di riepilogo appiccicata in alto.
- `Prepara i dossier mancanti`: genera in sequenza i pacchetti per tutti i selezionati, con avanzamento e conteggio dei falliti. Una chiamata per annuncio, in serie: il parallelo farebbe scattare il limite di richieste.
- Hub di invio per annuncio: Email (`mailto:`), WhatsApp (`wa.me`), Telegram (`t.me/share`), Copia tutto. All'uso di un canale lo stato passa a INVIATA e parte il follow-up a 3 giorni.
- AiSearch: costruisce i termini dai ruoli dichiarati (o, in mancanza, dai fatti di tipo esperienza non dedotti) e apre ricerche mirate su 5 portali.

Cosa NON e' stato costruito, e perche'

Richiesta	Stato	Motivo
Scansione automatica della rete	● impossibile ora	CORS e anti-bot bloccano il <code>fetch</code> dal browser. Richiede un Worker lato server e quindi il portatile
Invio automatico su portali/ATS	● impossibile ora	Serve automazione browser desktop
Allegati automatici (CV in PDF)	● limite di piattaforma	Nessuno schema <code>mailto: / wa.me / t.me</code> supporta allegati. Il CV va allegato a mano o incollato nel corpo
API WhatsApp/Telegram	🚧 radar	WhatsApp richiede la Business API a pagamento; Telegram richiede un bot lato server

Scelta tecnica sui link di ricerca. I portali non hanno un formato di query verificabile senza provarlo uno per uno, e inventarlo avrebbe prodotto link rotti. I link passano quindi da una ricerca Google con `site:` – formato certo, nessun parametro indovinato.

L'ultimo miglio resta umano. Nessun prodotto sul mercato invia candidature a portali da un telefono. Il massimo raggiungibile e' il pacchetto pronto piu' il canale precompilato piu' un tocco tuo. AiJob fa questo.

0.4.0 – dettaglio

Unificazione con il Registro candidature, che viveva in un'altra conversazione. Il file e' ora nominato `index.html` per essere caricato direttamente su hosting statico.

Vincolo di postazione come parametro, non come etichetta. Campo `Quanto puoi stare in piedi di fila` (15/30/60/120 minuti oppure nessun vincolo). Alimenta una nuova dimensione del punteggio di convenienza, `postazione_fissa`, con scala esplicita nel prompt: 100 interamente seduto, 50 misto con guardiola, 0 in movimento continuo.

Tre regole vincolanti nel prompt: il vincolo pesa sulla convenienza e va citato nel "perche"; se l'annuncio tace, va dedotto dalle mansioni e dichiarato; **non compare mai nei documenti generati ne' come condizione di salute**. Il dato resta nel dispositivo, non entra in CV, lettera o email.

Termini di ricerca. Dieci preimpostati come pulsanti, composti con la zona del profilo. Nessuno di essi nomina condizioni personali: in Svizzera restringerebbero i risultati a un

circuito che non riguarda un frontaliere. Il filtro utile e' la postazione, non la condizione.

Fasi allineate al Registro: DA CONTATTARE, INVIATA, RISPOSTA, COLLOQUIO, CHIUSA, con filtro e conteggio per fase.

Esporta CSV con separatore `;` e BOM UTF-8, apribile da Excel svizzero e italiano senza passaggi di importazione. Colonne: ID, azienda, ruolo, luogo, so farlo, mi conviene, postazione fissa, verdetto, fase, contatto, fonte, creato, follow-up.

Percorso verso l'accesso Google

Non serve il portatile: serve un'origine HTTPS. Le origini JavaScript autorizzate devono usare HTTPS (solo localhost e' esente) e non possono contenere percorso, query o frammento.

`file://` non e' registrabile: da cartella download OAuth non funzionera' mai.

Ordine corretto: pubblicare `index.html` su hosting statico gratuito con HTTPS → registrare l'origine → OAuth PKCE via Google Identity Services →

`drive.file` + `gmail.compose` + `calendar.events`. Tutto dal telefono.

Il portatile resta necessario solo per il Worker di AiSearch.

0.5.0 — dettaglio

Pubblicata su `https://robertonarcisojr.github.io/AiJob/`. L'origine HTTPS ha sbloccato OAuth.

Generatore PDF senza librerie. ~70 righe: Helvetica, A4, a capo automatico, impaginazione multipagina, codifica WinAnsi con rimappatura della punteggiatura tipografica (trattini lunghi, virgolette curve, puntini di sospensione). Verificato con `pypdf`: il testo si estrae corretto, accenti italiani inclusi; i caratteri fuori Latin-1 diventano `?` invece di corrompere il file. Funziona anche senza Google e senza rete.

Google. Token client di Google Identity Services — il modello supportato per un'app di solo browser. Il token vive **solo in memoria**, mai in `localStorage`, e scade in circa un'ora.

Scope	Cosa consente	Cosa NON consente
<code>gmail.compose</code>	creare bozze	leggere la posta, inviare da solo
<code>drive.file</code>	i soli file creati da AiJob	vedere il resto del Drive
<code>calendar.events</code>	aggiungere eventi	leggere l'agenda esistente

Bozza Gmail con allegati veri. Messaggio MIME multipart costruito nel browser: corpo in

UTF-8 base64, CV e lettera come PDF allegati, oggetto codificato RFC 2047 per gli accenti.

Risolve il limite di `mailto:`, che non può allegare nulla. Resta una bozza: l'invio è umano.

Drive. Salva CV, lettera e `dossier.json` con punteggi ed evidenze. Upload multipart.

Calendar. Evento colloquio con promemoria a 2 ore; lo stato passa a COLLOQUIO.

Limite noto. L'import del CV da Drive non c'è: `drive.file` dà accesso solo ai file creati dall'app, e leggere file esistenti richiederebbe uno scope di sola lettura che Google classifica come sensibile e sottopone a verifica. Il CV resta da incollare. Scelta deliberata: meno permessi.

0.6.0 — dettaglio

Diagnostica Google. Il collegamento falliva con un messaggio inutile. Due cause corrette:

1. La libreria Google è caricata con `async defer`: al primo tocco poteva non essere ancora pronta e l'app falliva subito. Ora attende fino a 4 secondi prima di dichiarare l'errore.
2. Nessuno strumento diceva cosa fosse rotto. Il pulsante **Diagnostica** verifica sette condizioni e le mostra in verde o rosso: indirizzo di apertura, protocollo, corrispondenza con l'origine registrata, stato della rete secondo il browser, caricamento della libreria, raggiungibilità di `accounts.google.com`, accesso alla memoria del browser.

Le due cause più probabili di un errore di rete apparente sono il blocco dei cookie di terze parti — Google richiede storage in iframe per l'autenticazione — e l'apertura del file da

`file://` invece che dal sito. La diagnostica distingue i due casi.

Analisi dei gap e formazione. Nuova sezione nell'AiBadge.

L'aggregazione è **deterministica in codice**: normalizza i requisiti, li conta sugli annunci reali, distingue senza prova da parziale, traccia le aziende. Nessun numero viene dal modello.

Il modello interviene solo dopo, per raggruppare i requisiti in temi formativi e proporre un'azione. Tre divieti nel prompt: copiare i requisiti alla lettera dall'elenco fornito, non produrre numeri, non inventare nomi di scuole o corsi. In codice, i gruppi che citano requisiti

inesistenti vengono **scartati**, e il conteggio delle richieste viene ricalcolato sommando i

dati reali. E' il primo punto del sistema in cui il Claim Firewall e' applicato dal codice e non dal prompt: un passo verso la chiusura di D-05.

Il modello puo' anche dichiarare su cosa **non** conviene investire.

0.7.0 — dettaglio

Diagnosi corretta. In 0.6.0 avevo attribuito il fallimento ai cookie di terze parti.

Sbagliato, o comunque non dimostrato: la libreria Google Identity Services usa FedCM proprio

per non dipenderne. La causa vera e' piu' semplice — **lo script**

`accounts.google.com/gsi/client`

non veniva caricato affatto. Brave lo blocca per impostazione predefinita, e qualunque blocco di contenuti puo' fare lo stesso su Chrome. Un errore mio di diagnosi, corretto da Roberto che ha provato Brave.

Rimedio strutturale: eliminata la dipendenza. AiJob non carica piu' alcuna libreria esterna. Il collegamento usa direttamente l'endpoint OAuth 2.0 di Google con

`response_type=token` e reindirizzamento a pagina intera:

- nessuno script di terze parti da bloccare
- nessuna finestra a comparsa: su mobile i popup vengono spesso soppressi
- `state` casuale generato con `crypto.getRandomValues` e verificato al ritorno
- frammento ripulito dall'indirizzo con `history.replaceState`: il token non resta nella barra
- token in `sessionStorage`, mai in `localStorage`: sparisce chiudendo la scheda

Ripresa dell'azione. Il reindirizzamento interrompe il flusso. Prima di partire, l'azione in corso viene messa in sospenso; al ritorno AiJob riapre l'annuncio giusto e la esegue da solo.

Diagnostica v2. Dieci controlli. Distingue i casi che prima si confondevano: file aperto da `file://`, indirizzo diverso da quello registrato, scudo del browser che blocca `accounts.google.com`, memoria di sessione inaccessibile.

Nota di metodo

Il flusso implicito e' quello che Google segnala come meno sicuro rispetto al codice con PKCE.

Lo scambio del codice richiede pero' un segreto lato server, e AiJob non ha un server. La scelta e' consapevole: token di breve durata, in memoria di sessione, con `state` verificato.

Da rivedere se e quando esistera' un backend.

0.7.1 — dettaglio

Adotta i due rilievi validi della revisione GPT, scarta quello superato.

Adottato: il pulsante non sparisce e l'errore ha un nome. La diagnostica ora apre con una riga **STADIO CHE FALLISCE** che nomina il primo controllo rosso, invece di lasciare dedurre. E un avviso permanente compare nel pannello Google, prima ancora di toccare qualcosa, quando l'ambiente non può funzionare.

Adottato: rilevamento della cornice. `window.top !== window.self`. Se AiJob gira dentro un'anteprima incorporata — per esempio il riquadro di anteprima di una chat — Google rifiuta l'accesso e nessuna configurazione lo cambia. Questa è la spiegazione più probabile del "Pagina web non disponibile" osservato aprendo l'app dentro Claude.

Scartato: il ritorno a GIS/FedCM. La revisione propone `AiJob -> GIS -> FedCM` e una diagnostica su `gis_script_loaded`, `fedcm_available`, `google_button_initialized`. È basata sulla 0.6.0: la 0.7.0 ha già rimosso quella libreria proprio perché era il componente che falliva. Reintrodurla significherebbe rimettere la dipendenza da uno script che Brave blocca per impostazione predefinita. FedCM risolve il problema dei cookie di terze parti, che non era il nostro; non risolve uno script che non viene mai scaricato.

Correzione a un limite dichiarato in 0.5.0

Avevo scritto che `drive.file` copre solo i file creati dall'app. È incompleto: combinato con il Google Picker, `drive.file` dà accesso anche ai file che l'utente sceglie esplicitamente. Il CV Master da Drive è quindi realizzabile senza scope sensibili.

Costo da valutare prima di farlo: il Picker richiede di caricare uno script da `apis.google.com` e una chiave API aggiuntiva. Rimetterebbe in gioco esattamente la classe di problema appena eliminata. Da decidere dopo che il collegamento base funziona.

0.7.2 — dettaglio

Revisione della proposta GLM. Tre elementi adottati, tre respinti.

Adottato — collegamento manuale. Se il reindirizzamento automatico non parte, un pulsante mostra l'indirizzo di autorizzazione da copiare o aprire in una scheda. Lo `state` viene

salvato

prima, quindi il ritorno funziona identico. E' l'ultima via d'uscita quando il browser blocca la navigazione automatica.

Adottato — rilevamento Brave, ma scritto correttamente. La proposta usava

`navigator.userAgent.includes("Brave")` : **Brave non si dichiara nello user agent**, si maschera

da Chrome per ragioni di privacy, quindi quel controllo e' sempre falso. Il rilevamento reale e'

`navigator.brave.isBrave()` , che restituisce una Promise. Implementato cosi', con l'avviso che compare quando la verifica si risolve.

Adottato — uscita dalla cornice. Non con `confirm` e `window.open` , che i browser mobili sopprimono, ma con un collegamento vero su cui toccare.

Respinto — token in `localStorage`. La proposta sostituiva `SS` con una versione che scrive

il token anche su `localStorage` come ripiego. Sarebbe una regressione: il token di accesso

finirebbe su disco, sopravvivendo alla chiusura della scheda. Adottato solo il ripiego **in memoria**, che non persiste nulla.

Respinto — `braveShieldBlocked()`. La funzione proposta crea un elemento `<a>` , non lo usa e

restituisce sempre `false` . Non verifica niente. E' la stessa classe di difetto della riga morta

trovata nello starfield: codice scritto e mai eseguito.

Respinto — `onclick` in linea con URL interpolato. `esc()` neutralizza l'HTML, non il contesto JavaScript: un apostrofo nell'URL romperebbe il gestore. Sostituito con `addEventListener` .

Avvertenza sul rapporto diagnostico

Il valore `STADIO CHE FALLISCE`: `Dentro una cornice` circolato tra gli agenti **non proviene da**

un'esecuzione reale. E' stato dedotto leggendo il codice, ed e' internamente contraddittorio:

dichiara di essere stato ottenuto aprendo il sito vero, ma "Dentro una cornice" puo' risultare

rosso solo se l'app e' dentro una cornice. Nessuno ha ancora eseguito la diagnostica sul telefono. Il dato resta mancante.

0.8.0 — dettaglio

Prima versione confermata funzionante sul telefono: CV, badge, Gemini, dossier, classifica.

Due difetti reali segnalati dall'uso.

403 su Gmail e Drive. Tre cause distinte, prima indistinguibili:

Causa	Come si riconosce	Rimedio
API non attiva nel progetto	<code>SERVICE_DISABLED</code> , "has not been used in project"	Attivare Gmail/Drive/Calendar API in Console
Permesso non concesso	<code>ACCESS_TOKEN_SCOPE_INSUFFICIENT</code>	Scollegare e ricollegare spuntando tutte le caselle
Utente non in lista di test	<code>access_denied</code>	Aggiungersi in Utenti di test

Il permesso concesso viene ora verificato contro l'endpoint `tokeninfo` , che restituisce gli scope realmente presenti nel token: se ne manca uno, AiJob lo dice per nome ("Bozze Gmail", "File su Drive") invece di riportare il corpo grezzo dell'errore.

Ricerca reale. Il limite dichiarato in 0.3.0 — CORS e anti-bot impediscono di leggere i portali dal browser — resta vero per il browser. Ma non si applica se a cercare e' il server di Google: la Gemini API supporta lo strumento `google_search` , che esegue la ricerca lato Google e restituisce `groundingMetadata` con le fonti reali. Nessun Worker, nessun backend, nessun CORS.

Vincoli applicati: lo strumento di ricerca non e' compatibile con la modalita' JSON forzata, quindi il formato e' richiesto nel prompt e il risultato viene interpretato in modo tollerante.

Le offerte prive di url o azienda vengono **scartate in codice**. Il prompt impone di preferire una lista vuota a un annuncio inventato, e l'interfaccia dichiara che i risultati non sono verificati uno per uno.

Limite noto: l'annuncio importato dalla ricerca contiene la sintesi, non il testo integrale. Per un'analisi accurata conviene aprire l'annuncio e incollare il testo completo.

0.9.0 — dettaglio

Punteggio opportunità. Le due domande restano separate, ma la classifica ora ordina su una terza grandezza calcolata **in codice, con pesi fissi e dichiarati all'utente**:

Componente	Peso
So farlo	34%
Mi conviene	28%
Freschezza	22%
Annuncio reale (100 – rischio fantasma)	16%

Il modello non decide questo numero: fornisce gli ingredienti, il calcolo e' aritmetica.

Freschezza. Fondata su dati verificati: un annuncio riceve la maggior parte delle candidature nelle prime 48-72 ore, e chi si candida entro 24-48 ore ottiene 2-3 volte piu' colloqui di chi aspetta una settimana. Scala: 0-2 giorni = 100, entro 7 = 82, entro 14 = 60, entro 30 = 35, oltre = 15. Data ignota = 50, valore neutro che non premia ne' punisce.

Rischio annuncio fantasma. Tra il 20% e il 40% degli annunci online sono posizioni che nessuno intende riempire. Il modello valuta segnali concreti — azienda non identificabile, descrizione senza mansioni, nessun contatto, retribuzione assente con requisiti vaghi, promesse sproporzionate, richieste di denaro — e li elenca. Un annuncio sospetto scende in classifica anche con fit alto.

Budget della ricerca — rimedio al 429. Chiave separata `aijob.search.v1` .

Regola	Valore
Ricerche al giorno	20
Intervallo minimo	6 secondi
Cache per query	4 ore

Ripetere la stessa ricerca entro quattro ore **non consuma quota**. Gli errori 429 vengono letti:

`retryDelay` e identificativo della quota compaiono nel messaggio.

Una sola chiamata per ricerca. La proposta di tre round automatici e' stata respinta: con una quota gia' esaurita, moltiplicare le chiamate peggiora il problema che dovrebbe risolvere.

Le offerte vengono deduplicate per azienda e ruolo normalizzati e ordinate per freschezza.

Backup su Drive. Salvataggio e ripristino dell'intero stato in `aijob-backup.json` . Il file esistente viene aggiornato con PATCH invece di duplicato. Il ripristino chiede conferma e verifica la struttura prima di sovrascrivere. Usa `drive.file` : AiJob vede solo il proprio file.

Nota su una dichiarazione non verificata

Un agente ha riferito di aver applicato quattro modifiche al file `index.html` . Nessun agente ha accesso a questo filesystem: le modifiche descritte non erano state applicate. Le idee valide di quella proposta — diagnosi del 429, budget persistente, cache, deduplica — sono state implementate qui, misurate e testate. La ricerca multi-round e' stata scartata con motivazione.

0.9.1 — dettaglio

Diagnosi della quota esaurita. Cambiare chiave non aiutava, e la ragione e' documentata: piu' chiavi dello **stesso progetto Google condividono la stessa quota**. La chiave autentica, il progetto possiede il limite. Se la seconda chiave nasce nello stesso progetto, non cambia nulla.

Seconda causa, piu' seria: **la ricerca con Google (grounding) e' sostanzialmente una funzione del piano a pagamento** — le prime 1.500 richieste al giorno sono gratuite *sui piani a pagamento*. Esiste inoltre un difetto documentato per cui le richieste con grounding consumano la quota `generate_content_free_tier_requests` (20 al giorno) invece della quota di ricerca, anche con chiavi di livello superiore. Su piano gratuito il 429 e' quindi atteso, non anomalo.

Rimedio: motore di ricerca separato e intercambiabile.

Motore	Endpoint	Modello
Gemini	grounding con Google Search	<code>gemini-3.6-flash</code>
Perplexity Sonar	<code>https://api.perplexity.ai</code>	<code>sonar</code>

Motore	Endpoint	Modello
OpenRouter	<code>https://openrouter.ai/api/v1</code>	<code>perplexity/sonar</code> o qualunque modello con <code>:online</code>

Entrambe le alternative parlano il formato Chat Completions di OpenAI e **cercano davvero nel web**. Le citazioni vengono lette sia da `annotations[].url_citation` sia dal campo `citations`.

Quando la ricerca Gemini fallisce per quota o per 403, e un motore alternativo e' configurato, AiJob **passa all'altro da solo** e lo dichiara. La ricaduta riguarda solo la ricerca: l'analisi degli annunci continua a usare il provider principale.

Semantica corretta. "Cerca davvero in rete" era brutto e fuorviante: descriveva l'implementazione invece del risultato. Ora "Trova opportunità" e il pulsante "Cerca opportunità".

0.9.2 — dettaglio

Prova ATS. Sei famiglie (generico, Workday, Greenhouse, Lever, SmartRecruiters, SuccessFactors) con pesi diversi sugli stessi sei criteri: contatti, titoli di sezione, formato delle date, risultati quantificati, parole chiave, pulizia del testo. **Nessuna chiamata AI:** sono regole in codice, quindi il risultato e' riproducibile e verificabile.

Se manca l'annuncio di riferimento, il peso delle parole chiave viene ridistribuito sugli altri criteri invece di contare come zero — altrimenti ogni CV verrebbe penalizzato per un dato assente.

Le correzioni applicabili sono solo quelle che **non toccano i fatti**: pulizia dei caratteri, titoli di sezione standard, formato delle date. Quantificare i risultati e aggiungere parole chiave restano azioni umane, marcate come tali e non spuntabili. Claim Firewall rispettato.

Difetto trovato dai test. La prima versione leggeva `2019 2024` come numero di telefono: qualunque coppia di anni faceva risultare i contatti presenti. Ora un telefono richiede almeno nove cifre e non deve essere una coppia di anni.

Scheda Info. Mappa dell'ecosistema con AiSecretary **riposizionato**: non e' un modulo dentro AiLife, e' uno strato che attraversa i domini. Vedi nota architetturale sotto. Include anche

lo
stato reale della versione in uso: annunci, fatti, motori configurati, ricerche consumate.

Nota architettuale — posizione di AiSecretary

L'osservazione di Roberto e' corretta. AiSecretary sotto AiLife, come fratello di AiEvents, non regge: la prima cosa che deve coordinare e' una candidatura, che vive in AiWork. Un modulo non puo' orchestrare un dominio di cui non fa parte.

Posizione corretta: **sopra i domini, sotto AiWorld Core**. AiWorld governa (policy, memoria, evidenze); AiSecretary opera (osserva, decide, prepara, chiede, ricorda); AiJob, AiEvents e gli altri forniscono le capacita'. AiSecretary non possiede funzioni proprie: le coordina.

Nota sul monolite

AiJob e' gia' un monolite: 1900 righe in un file solo, e funziona. Il problema non e' la dimensione ma la verificabilita': finche' ogni funzione ha un test e ogni numero ha un calcolo ispezionabile, il file puo' crescere. Il giorno in cui una modifica ne rompe un'altra senza che i test se ne accorgano, quello e' il segnale per separare — non prima.

0.10.0 — dettaglio

Oggi: **la homepage decide al posto tuo**. Non e' un riassunto: e' una coda di azioni ordinate per urgenza, calcolate dai dati reali senza alcuna chiamata AI.

Priorita'	Cosa genera l'azione
0	Configurazione mancante: chiave del motore, AiBadge assente
1	Follow-up scaduto, colloquio in agenda
2	Dossier gia' pronto non inviato; opportunita' entro la finestra di 72 ore
3	Lacuna che ricorre in piu' annunci; fatti dedotti da confermare

Ogni azione porta esattamente dove serve: apre l'annuncio, espande il dettaglio, scorre al punto giusto. A dati vuoti mostra i tre passi iniziali con le spunte gia' fatte.

Stato condiviso AiWorld — l'Hub senza un secondo repository. Un file `aiworld-state.json`

su Drive dove ogni modulo scrive la propria sezione: versione, numeri, azioni aperte, indirizzo.

Il meccanismo che lo rende possibile: `drive.file` concede accesso ai file "creati o aperti con questa app", e l'ambito e' legato al **Client ID**, non al dominio. AiEvents che usa lo stesso Client ID e' la stessa app agli occhi di Drive, quindi legge e scrive lo stesso file. Nessun server, nessuna duplicazione di dati, nessuna pagina che finge funzioni inesistenti. [Affidabilita': Media — meccanismo documentato, da confermare con AiEvents in esecuzione]

Questa e' la ragione per cui **non ho creato un repository Hub separato**: una mappa statica in un terzo posto sarebbe una terza copia da tenere aggiornata a mano. Lo stato condiviso e' la sostanza dell'Hub; la mappa vive gia' nella scheda Info.

Prossimo passo per AiEvents

1. Aggiungere `https://robertonarcisojr.github.io` come origine autorizzata (se il repository e' diverso, aggiungere il suo indirizzo di ritorno).
2. Portare tre blocchi cosi' come sono: OAuth per reindirizzamento, generatore PDF, adapter multi-provider.
3. Scrivere la sezione `moduli.aievents` nello stesso file di stato.

A quel punto la scheda Info di AiJob mostrera' AiEvents senza che nessuno abbia scritto una riga di collegamento fra le due app.

0.11.0 — dettaglio

Il blocco vero. La ricerca restituisce quasi solo annunci con candidatura dal portale dell'azienda. Tutto il percorso costruito finora finiva in una bozza Gmail che in quei casi non serve: alcune aziende dichiarano esplicitamente che le candidature via email non vengono prese in considerazione.

Canale di candidatura. L'analisi ora classifica ogni annuncio: EMAIL, PORTALE, AGENZIA, SCONOSCIUTO. Il canale compare in classifica accanto alla freschezza, e le istruzioni dell'annuncio vengono copiate alla lettera invece di essere riassunte.

Kit portale. Per gli annunci PORTALE e AGENZIA compare un percorso diverso dal dossier email: testi gia' dimensionati per i campi di un modulo online, ciascuno con il proprio pulsante Copia.

Campo	Limite
Lettera breve	1000 caratteri
Motivazione	400 caratteri
Punti di forza	tre righe, ognuna sostenuta da un fatto
Disponibilita'	dedotta dai vincoli, altrimenti "da concordare"
Domande frequenti	risposte pronte, 300 caratteri

Il kit dichiara anche cosa **non** puo' compilare, invece di inventarlo. Il CV si scarica in PDF e lo carichi tu: nessun modulo online accetta allegati da un'altra pagina.

Nome del marchio. Corretto ovunque compaia all'utente: **AiJob** con la J maiuscola, incluso il nome del modulo nello stato condiviso e il file di backup scaricato. Restano in minuscolo solo le chiavi interne di memoria, che nessuno vede.

0.12.0 — dettaglio

La separazione che risolve il 429. La quota bloccata e' solo quella del *grounding*: le chiamate normali a Gemini continuano a funzionare, come dimostra il fatto che badge, dossier e kit vengono generati senza errori. Finora ricerca e struttura viaggiavano nella stessa richiesta, quindi la struttura moriva insieme alla ricerca.

Ora sono due passaggi distinti:

Passo	Chi lo fa	Quota consumata
Trovare le pagine	Tavily	1 credito su 1000 mensili
Capire quali sono offerte vere	provider principale, chiamata normale	quota ordinaria, non quella di ricerca

E' capability routing applicato al caso concreto, non in astratto: la ricerca e il ragionamento sono capacita' diverse e non devono dipendere dallo stesso limite.

Tavily. 1000 ricerche al mese, senza carta di credito, chiave da **tavily.com**. Nel motore di ricerca compare come quarta opzione e non richiede di scegliere un modello.

Limite non verificabile da qui: non so se Tavily accetti chiamate diritte dal browser. Se le rifiuta per CORS, l'app ora lo dice esplicitamente invece di mostrare un errore generico — si scopre in trenta secondi provando. Perplexity e OpenRouter restano le alternative.

Registro agenti nella scheda Info. Cinque provider con capacita' dichiarate, ruolo assegnato (analisi o ricerca) e stato: *non configurato, configurato ma non provato, risponde, non risponde*. Lo stato non e' dichiarato dal codice: si ottiene provando. Per i non configurati c'e' il collegamento diretto alla console dove ottenere la chiave.

E' il primo mattone estraibile verso AiAgents: registro, capacita', ruolo, prova, ricaduta. Vive dentro AiJob perche' il gate viene prima; quando sara' provato, si estrae.

0.13.0 — dettaglio

Il difetto. Chi voleva usare una chiave OpenAI o OpenRouter riceveva "manca la Base URL del provider". Chiedere un indirizzo tecnico a chi vuole solo incollare una chiave e' un errore di progetto, non un campo mancante. Ed era aggravato da una seconda scelta sbagliata: due configurazioni separate, una per l'analisi e una per la ricerca, con menu diversi.

Sostituite entrambe da un registro unico. Otto provider a catalogo, ognuno con indirizzo, modello predefinito, capacita' e link alla console gia' noti all'app.

Provider	Ragiona	Cerca	Indirizzo
Google Gemini	si	con grounding	noto
OpenAI	si	no	api.openai.com/v1
OpenRouter	si	con :online	openrouter.ai/api/v1
Groq	si	no	api.groq.com/openai/v1
DeepSeek	si	no	api.deepseek.com
Perplexity Sonar	si	nativa	api.perplexity.ai
Tavily	no	solo ricerca	noto
Altro compatibile	si	no	lo indichi tu

L'unico che chiede un indirizzo e' "Altro". Tutti gli altri: chiave, eventualmente modello, fine.

Due catene con ricaduta automatica. Ragionamento e ricerca sono capacita' distinte, ognuna con la propria fila. Il primo prova per primo; se e' a quota o non risponde, tocca al successivo e AiJob lo dichiara. L'ordine si cambia con un tocco: "usa per primo".

Ragionamento: Gemini → OpenRouter → Groq → DeepSeek → OpenAI → Perplexity
Ricerca: Tavily → Perplexity → OpenRouter → Gemini

Solo i provider con una chiave entrano in fila. Questo e' l'Agent Control Plane proposto da GPT, ridotto a cio' che serve: registro, capacita', ordine, prova, ricaduta.

Migrazione automatica. Le chiavi gia' inserite nelle vecchie schermate vengono trasferite nel registro al primo avvio, una sola volta. Nessuna riconfigurazione.

Ripiego sul formato JSON. Alcuni modelli rifiutano `response_format: json_object` con un 400. La chiamata viene ripetuta senza quel parametro invece di fallire.

0.13.1 — due regressioni introdotte da me in 0.13.0

Entrambe riprodotte in ambiente controllato prima di correggerle, non ipotizzate.

1. Tre pulsanti scollegati. Ripulendo i gestori della vecchia schermata ho tagliato un blocco di codice troppo largo, portandomi via anche `Collega Google`, `Scollega` e il nuovo `Prova gli agenti`. I pulsanti c'erano, non facevano nulla, e nessun errore compariva in console: un ascoltatore mancante e' silenzioso. Reinseriti tutti e tre.

Alla riparazione e' emerso un secondo difetto nello stesso blocco: una costante riassegnata dentro il ciclo, che interrompeva la prova dopo il primo agente.

2. Il contatore che bloccava senza motivo. Il limite locale veniva incrementato **prima** della chiamata. Ogni tentativo fallito per quota contava come ricerca eseguita: venti errori consecutivi producevano venti ricerche mai avvenute, e poi il blocco.

Tre correzioni:

Prima	Ora
Contava i tentativi	Conta solo le ricerche riuscite
Contatore unico globale	Conteggio separato per provider
Bloccava a 20	Non blocca: avvisa e prosegue, la quota vera la decide il provider

Chiave del contatore cambiata in `aijob.search.v2`: il conteggio sbagliato accumulato non viene ereditato. Aggiunto il pulsante **Azzera il contatore locale** nella scheda Agenti.

Errore visibile per agente. Quando la prova fallisce, il motivo compare sotto il nome del provider invece di sparire in un avviso temporaneo.

0.14.0 — dettaglio

Sfondo. Portato da `#12161c` a `#06070a`: quasi nero, contrasto testo/sfondo da 15.8 a 17.6.

Non nero assoluto, e la ragione e' misurabile piu' che estetica: su schermi OLED il nero puro accanto a testo chiaro produce sfocatura ai bordi delle lettere, e il passaggio brusco fra pixel spenti e accesi affatica in lettura prolungata. Un grigio quasi nero elimina l'effetto e conserva gran parte del risparmio di batteria. Tutti i dodici contrasti restano a norma.

OpenAI non e' chiamabile da una pagina web. Non e' una chiave sbagliata ne' un difetto di AiJob: `api.openai.com` non autorizza le richieste dirette dal browser. Ora il provider e' marcato `browser: false`, resta fuori dalle catene anche se ha una chiave, e la sua scheda lo dichiara con l'alternativa: gli stessi modelli passano da OpenRouter.

Diagnostica per codice di stato. Prima 401 e 403 producevano lo stesso messaggio ambiguo "chiave non valida o senza credito", che non permette di capire cosa fare.

Codice	Messaggio
401	chiave rifiutata: controlla di averla copiata per intero
402	credito esaurito: ricarica o usa un modello <code>: free</code>

Codice	Messaggio
403	accesso negato a questo modello con la tua chiave
404 + "data policy"	il modello e' escluso dalle impostazioni privacy di OpenRouter
404	il modello non esiste: controlla il nome
429	limite di richieste, marcato come quota e quindi con ricaduta
5xx	il servizio non risponde, riprova

Il caso `data policy` merita una nota: su OpenRouter i modelli gratuiti richiedono di autorizzare l'uso dei prompt per l'addestramento in `openrouter.ai/settings/privacy`. Senza quel permesso tornano 404 con un messaggio che sembra un errore di modello inesistente.

Intestazioni OpenRouter. Aggiunte `HTTP-Referer` e `X-Title`, che OpenRouter usa per identificare l'applicazione chiamante.

0.15.0 — dettaglio

Correzione a una diagnosi altrui. Per DeepSeek era stato ipotizzato un blocco CORS. Il messaggio ricevuto lo smentisce: "credito esaurito" nasce da un HTTP 402, quindi la richiesta e' arrivata al server ed e' stata risposta. Se fosse stato CORS, il messaggio sarebbe stato "il browser non e' riuscito a contattare il servizio". DeepSeek accetta le chiamate dal browser ma **non ha un piano gratuito**: una chiave nuova ha saldo zero. Nota del provider corretta.

Lo stesso vale per OpenRouter: 402 significa account senza credito, non chiave sbagliata.

Scelta dei modelli gratuiti dentro l'app. L'elenco `openrouter.ai/api/v1/models` e' pubblico e non richiede chiave. Il pulsante *Mostra i modelli gratuiti* lo legge, filtra quelli con prezzo zero o suffisso `: free`, e li presenta come pulsanti: un tocco riempie il campo modello. Niente piu' ricerche a mano nel sito.

Autodiagnosi e autocorrezione. Nella scheda Info, quindici controlli deterministici su cinque aree: agenti, profilo, candidature, memoria, Google. Nessuna chiamata AI — le regole sono nel codice e chiunque puo' rileggerle.

Esempi di cio' che rileva: catena senza ricaduta, provider con chiave ma inutilizzabile dal browser, fatti marcati come documentati ma privi della frase che li sostiene, candidature senza punteggio o senza registro evidenze, memoria vicina al limite, cache di ricerca gonfia.

Tre problemi hanno una correzione applicabile con un tocco:

Problema	Correzione
Fatto "dal CV" senza fonte	riportato a DEDOTTO, torna a te confermarlo
Candidatura senza punteggio	campi normalizzati a zero e avviso di rianalizzare
Cache di ricerca gonfia	svuotata

Il principio e' lo stesso del resto del progetto: la correzione **non inventa il dato mancante**. Riporta il sistema a uno stato onesto e dice cosa serve per riempirlo davvero.

Il numero di problemi rilevati entra in `aiworld-state.json`: quando altri moduli faranno lo stesso, l'autodiagnosi diventa una proprieta' di AiWorld, non di AiJob.

Nota sul nome del file

Il file **deve** chiamarsi `index.html` perche' GitHub Pages serva `robertonarcisojr.github.io/AiJob/`. Rinominarlo in `AiJob.html` cambierebbe l'indirizzo in `.../AiJob/AiJob.html` e romperebbe l'indirizzo di ritorno registrato in Google Cloud, che e' esatto. L'archivio scaricato si chiama ora `AiJob-0.15.0.zip`; dentro resta `index.html`.

0.16.0 — la ricerca smette di essere un motore di ricerca

Il problema vero, dichiarato da Roberto: se per avere annunci deve cercarli a mano sui portali e incollarli uno a uno, l'app non ha ragione di esistere. Corretto.

Tutti i motori configurati finora — Gemini grounding, Tavily, Perplexity — cercano **pagine web**.

Restituiscono risultati che un modello deve poi interpretare, con tre difetti: consumano quota di ragionamento, sbagliano l'interpretazione, e non conoscono la data di pubblicazione.

Adzuna e' un indice di annunci, non un motore di ricerca. Copre la Svizzera (`/ch/`), ha un piano gratuito di circa 1000 chiamate al mese, e restituisce record gia' strutturati:

Campo Adzuna	Uso in AiJob
<code>title</code>	ruolo
<code>company.display_name</code>	azienda
<code>location.display_name</code>	luogo
<code>created</code>	giorni dalla pubblicazione, calcolati sulla data vera
<code>redirect_url</code>	link all'annuncio
<code>description</code>	sintesi
<code>salary_is_predicted</code>	avviso "salario stimato, non dichiarato"

Conseguenza importante: **una ricerca non consuma quota di ragionamento**. L'elenco arriva già pronto. Il modello serve solo dopo, per confrontare gli annunci scelti con l'AiBadge.

Parametri fissati: `max_days_old=30` , `sort_by=date` , venti risultati. Gli annunci senza link vengono scartati in codice.

Analizza i primi 8. Dal risultato della ricerca, un pulsante manda gli annunci direttamente in classifica: estrazione, punteggio, evidenze, verdetto, uno dopo l'altro, con avanzamento visibile. Se la quota di ragionamento finisce a meta', si ferma e dichiara quanti ne ha fatti invece di fallire in silenzio. La data di Adzuna non viene sovrascritta da una deduzione del modello.

Da qui il percorso è: apri AiSearch → Cerca opportunità → Analizza i primi 8 → la classifica è piena. Nessun incollaggio.

Credenziali. Adzuna richiede due valori, `app_id` e `app_key` , e il registro ora gestisce i provider a doppia credenziale: senza entrambi il provider non entra in catena.

Limite non verificabile da qui: non so se Adzuna invii le intestazioni CORS necessarie alle chiamate dal browser. Se le rifiuta, l'app lo dice con il messaggio dedicato. Si scopre in un minuto registrandosi su `developer.adzuna.com/signup` .

0.17.0 — il Radar, e perché era l'unica strada

Ammissione preliminare. La ricerca non ha mai funzionato davvero, e la causa non erano i provider: era il posto da cui la eseguivo. Una pagina web può chiamare solo i servizi che autorizzano esplicitamente le chiamate dal browser. jobs.ch, ticinolavoro e Job-Room non lo fanno e non hanno API pubbliche. Ogni motore aggiunto — Gemini grounding, Tavily, Perplexity, Adzuna — girava attorno a quel limite invece di risolverlo, e ognuno portava una quota o una chiave in più.

La soluzione: spostare la ricerca su un server che già possiedi. GitHub Actions esegue codice gratuitamente sui repository pubblici, su pianificazione. Da lì non esiste il problema delle chiamate dal browser: è un server che interroga un altro server.

```
GitHub Actions, ogni mattina alle 6:10
    ↓
scripts/cerca.mjs interroga Adzuna
    ↓
deduplica, scarta i vecchi, ordina per data
    ↓
scrive annunci.json nel repository
    ↓
AiJob lo legge dalla stessa origine
```

Conseguenze: **nessuna chiave da configurare in AiJob, nessuna quota consumata all'apertura, nessun limite del browser.** Il Radar è il primo motore della catena ed è sempre pronto.

Cosa fa lo script. Interroga Adzuna per ogni combinazione di ruolo e zona indicata in `ricerca.json`, con pausa fra le chiamate. Deduplica su azienda, ruolo e luogo normalizzati. Conserva lo storico, così un annuncio trovato ieri resta anche se oggi non compare più nei risultati. Scarta ciò che supera i giorni impostati. Se la quota Adzuna finisce a metà, si ferma e salva quello che ha invece di perdere tutto.

Verificato in locale con risposte simulate: deduplicazione, scarto dei vecchi, normalizzazione degli spazi, ordinamento per data, marcatura del salario stimato.

Lato AiJob. `cercaRadar` legge `annunci.json` con `cache:"no-store"`, filtra per ruoli e zona del profilo — e se il filtro non lascia nulla mostra tutto invece di una schermata vuota — calcola i giorni sulla data reale e dichiara quando l'archivio è stato aggiornato. Se il file non c'è, il messaggio non è un errore ma l'istruzione per attivarlo.

Sequenza dei motori: Radar, Adzuna, Tavily, Perplexity, OpenRouter, Gemini.
Il primo che risponde vince; gli altri restano come riserva.

0.17.1 — Adzuna esce dal percorso critico

Il rilievo di GPT e' fondato: la registrazione ad Adzuna chiede nome e sito dell'organizzazione, e l'accesso viene concesso a discrezione. Non e' una richiesta adatta a chi cerca lavoro. Va tolta dal percorso obbligatorio.

Con una precisazione utile: quei campi non richiedono una societa'. "AiJob — progetto personale" e l'indirizzo `robertonarcisojr.github.io/AiJob/` sono risposte vere e sufficienti. Vale il tentativo, ma **non deve essere il presupposto perche' il Radar funzioni**.

Radar v2: sistema di adattatori. Le fonti non sono piu' scritte nel codice ma elencate in `fonti.json`. Due adattatori:

Tipo	Chiave	Cosa legge
<code>rss</code>	nessuna	qualunque feed pubblicato: portali minori, pagine carriere aziendali, aggregatori
<code>adzuna</code>	due segreti del repository	indice Adzuna, facoltativo

Aggiungere una fonte e' una voce in un file, non una modifica al programma. Una fonte che smette di rispondere non ferma le altre: viene marcata nel rapporto e il giro continua.

Perche' RSS. Non esistono API pubbliche per jobs.ch, Job-Room o ticinolavoro — verificato.

Ma molte pagine carriere aziendali e portali minori pubblicano un feed, e consumare un feed pubblicato e' esattamente l'uso per cui esiste. Nessuna zona grigia, nessuna fragilita' da scraping.

Il lettore RSS gestisce sia `item` che `entry`, estrae il link anche dall'attributo `href`, ripulisce tag HTML ed entita', scarta le voci senza link o titolo. Verificato con feed simulati, inclusi un feed irraggiungibile e uno con voci incomplete.

In AiJob il rapporto per fonte compare sotto i risultati: si vede subito quale ha prodotto annunci e quale non ha risposto.

Cosa resta vero

Nessuna delle fonti disponibili copre bene gli annunci ticinesi. Il Radar rende il problema **estendibile** invece che bloccato: ogni feed trovato si aggiunge in trenta secondi. Ma non promette una copertura che oggi non esiste.

0.18.0 — Radar v3: l'appiglio stabile

Perche' tutti i tentativi precedenti erano fragili. API a pagamento o con registrazione aziendale, feed RSS che i portali svizzeri non pubblicano, selettori CSS che si rompono al primo restyling. Nessuno di questi e' un contratto che il portale ha interesse a mantenere.

Ne esiste uno. Google richiede a ogni pagina annuncio cinque proprieta' in JSON-LD — `title`, `description`, `datePosted`, `hiringOrganization`, `jobLocation` — perche' l'annuncio compaia in Google for Jobs. Un portale che smette di pubblicarle sparisce dai risultati di Google. E' il dato piu' stabile che esista su quelle pagine, ed e' gia' machine-readable.

L'adattatore `jsonld`. Due strategie in cascata:

1. Legge i JSON-LD della pagina di ricerca. Molti portali vi includono un `ItemList` con tutti gli annunci: in quel caso basta una sola richiesta.
2. Se non bastano, estrae i link agli annunci (filtrati per pattern e limitati allo stesso dominio) e legge ogni pagina.

Il lettore attraversa l'albero JSON in profondita', quindi trova i `JobPosting` ovunque siano annidati, gestisce `@type` come stringa o come elenco, ripulisce HTML ed entita', compone il luogo da `addressLocality` e `addressRegion`, normalizza le date.

Aggiungere un portale non richiede codice: una voce in `fonti.json` con l'indirizzo di una pagina di ricerca. Sono precaricati `jobs.ch`, `JobScout24` e `Carriera.ch` con i filtri per il Ticino.

Rapporto diagnostico per fonte. Ogni fonte riporta stato della pagina, byte ricevuti, quanti JSON-LD trovati, quanti link candidati, quante pagine aperte, quante contenevano dati strutturati, quale strategia ha usato. Visibile nella scheda Info dell'app.

Questo e' il punto: se una fonte non funziona, il rapporto dice **perche'**, e la si sostituisce in trenta secondi invece di indovinare.

Verificato con due portali simulati di struttura diversa piu' uno irraggiungibile: filtro dei link, esclusione dei domini esterni, lettura di `ItemList`, pulizia del testo, resilienza.

Su Swiss Job Hunter e AGPL

Il progetto risolve lo stesso problema con Python, Playwright e scraper per portale, sotto licenza AGPL-3.0. Non ho riusato il suo codice: la strada JSON-LD ottiene lo stesso risultato con meno righe, senza browser headless, senza vincoli di licenza e senza dipendere da selettori che qualcun altro deve mantenere. Resta un ottimo riferimento se il JSON-LD dovesse rivelarsi assente sui portali che ti servono.

0.18.1 — Radar v4: le due fonti che contano

Verificato, non riferito. Un agente ha citato endpoint precisi dichiarando di aver letto un repository. Ho clonato quel repository ([Donvink/swiss-job-hunter](#) , AGPL-3.0, 137 stelle) e letto il codice. Gli endpoint esistono davvero:

Portale	Endpoint	Parametri	Risposta
jobs.ch	<code>www.jobs.ch/api/v1/public/search/</code>	<code>query , location , page , num_pages , sort=date</code>	<code>documents[] , total_hits</code>
jobup.ch	<code>job-search-api.jobup.ch/search</code>	<code>term , location , page , rows + intestazioni Origin e Referer</code>	<code>documents[] , numPages</code>

Verificata anche la correzione piu' importante: **jobs.ch e jobup.ch non pubblicano JSON-LD nelle**

pagine di ricerca. La strada JSON-LD della 0.18.0 funziona per JobScout24 e Carriera.ch, ma da

sola avrebbe escluso i due portali piu' grandi — e jobup copre il Ticino. Il Radar doveva essere ibrido.

Sul riuso. Non ho copiato codice AGPL. Ho letto quali indirizzi rispondono e come si chiamano

i campi: sono fatti sul protocollo, non espressione protetta. L'implementazione in JavaScript e'

originale, con scelte diverse — nessun browser headless, nessun modello semantico,

nessun
database, nessun server.

Mappature ricostruite e verificate con risposte simulate fedeli:

- jobs.ch: identificativo estratto come UUID dallo slug, che spesso porta il titolo in coda;
luogo composto da `place` e prima regione; percentuale di impiego da `employment_grades`,
che diventa la nota "impiego 80–100%" in classifica.
- jobup.ch: `place` arriva come stringa oppure come oggetto con `name` — gestiti entrambi;
indirizzo costruito da `slug` o, in mancanza, da `id`.
- Entrambi: voci senza titolo o senza identificativo scartate in codice.

Ordine delle fonti: jobs.ch, jobup.ch, JobScout24, Carriera.ch, Adzuna (spenta).
Ogni fonte riporta chiamate, errori, ultimo stato HTTP e totale dichiarato dal portale.

Avvertenza onesta

Questi endpoint non sono API pubblicate con un contratto: funzionano oggi e potrebbero cambiare.

Per questo il Radar interroga cinque fonti e nessuna e' indispensabile, e per questo il rapporto

per fonte esiste: quando una smette di rispondere si vede subito quale, e le altre continuano.

0.18.2 — togliere le barriere invece di aggiungere funzioni

La domanda era: si puo' fare senza tutta questa tecnica? La risposta era si', e il problema era mio: chiedevo cinque file, due segreti di repository e una registrazione a un servizio.

	Prima	Ora
File da caricare	5	1
Segreti da creare	2	0
Registrazioni	1 (Adzuna)	0
Cartelle da creare	2	0, il percorso nel nome le crea
Passi	5	3

Come. Lo script del Radar vive dentro il file del workflow. Il primo passo lo scrive su disco, il secondo lo esegue, l'ultimo lo cancella. Ruoli e zone sono scritti nel file con valori

predefiniti già adatti al caso, e si possono cambiare anche senza toccare nulla:
comparendo
come campi quando si avvia il workflow a mano.

Adzuna è stata rimossa del tutto. Le quattro fonti che restano — jobs.ch, jobup.ch, JobScout24, Carriera.ch — non chiedono nulla a nessuno.

Verifica. Il file YAML è valido, e lo script estratto dal workflow è stato eseguito separatamente con risposte simulate: stesso risultato del file originale, tre fonti su quattro rispondono, la quarta viene segnalata con il suo codice di errore invece di far fallire il giro.

Cosa resta tecnico, e perché

Restano le chiavi degli agenti AI, che servono per analizzare gli annunci e scrivere i documenti.
Quella parte non si può togliere senza mettere una chiave mia dentro l'app, che sarebbe a carico mio e visibile a chiunque. Ma la ricerca — la parte che non ha mai funzionato — adesso non chiede più niente.

0.19.0 — la ricerca funziona, si chiude il resto del ciclo

Primo giorno in cui AiSearch restituisce annunci veri, con Tavily configurato.

Diagnostica che prova invece di dichiarare. Prima verificava solo che il token esistesse: tutto verde mentre il salvataggio falliva. Ora esegue quattro chiamate reali — bozze Gmail, elenco Drive, lettura calendario e **scrittura sul calendario**, con l'evento di prova rimosso subito dopo. La scrittura è il punto in cui il calendario fallisce più spesso, e in lettura non si vede. Gli errori vengono tradotti: "API non attiva, aprila nella Libreria della Console" invece del corpo grezzo.

Invio reale. Aggiunto lo scope `gmail.send`, che Google classifica come *sensibile* e non *ristretto*: nessun audit di sicurezza. Il pulsante **Invia adesso** compare solo quando esiste un destinatario, chiede conferma mostrando destinatario, oggetto e allegati, dichiara che l'azione non è annullabile. Dopo l'invio lo stato passa a INVIATA e parte il follow-up.

Monitoraggio della posta: non implementato, e la ragione non è tecnica. Leggere la posta richiede scope *ristretti*, e Google impone a chi li usa un assessment CASA annuale da alcune centinaia a diverse migliaia di dollari, con revalidazione ogni dodici mesi. Per un'app

personale
non ha senso. Lo stato delle candidature resta manuale.

Distanza, senza chiavi. Geocodifica via Nominatim di OpenStreetMap, distanza con la formula
dell'emiseno verso, maggiorazione 1.35 per il percorso reale, stima dei minuti a 55 km/h,
piu' il
link al percorso vero su OpenStreetMap. Le coordinate vengono messe in cache: la
stessa citta'
non viene chiesta due volte. Nuovo campo "da dove parti" nel profilo.

Video. Nella sezione formazione, ogni tema apre una ricerca YouTube mirata. Nessuna
chiave,
nessun dato inviato: e' un collegamento. L'API YouTube servirebbe solo per mostrare i
risultati
dentro l'app, e non vale una chiave in piu'.

Sul nome AiRadar

Il Radar e' gia' il nome del componente che raccoglie gli annunci, ed e' il nome giusto:
descrive
cosa fa. Promuoverlo a modulo separato — **AiRadar** accanto ad AiJob — creerebbe
pero' lo stesso
problema di AiBridge: due nomi per una cosa sola, e la domanda "dove sta la ricerca?"
senza una
risposta unica. Resta il **Radar di AiJob**, minuscolo, componente. Se un giorno AiEvents o
un
altro modulo useranno lo stesso motore, allora avra' senso promuoverlo — e in quel
momento avra'
anche un utente in piu' che lo giustifica.

0.19.1 — marchio

Perche' non il simbolo proposto. Il simbolo dei Doni della Morte e' marchio registrato di
Warner Bros. Una delle registrazioni copre esplicitamente *servizi di negozio online e
gestione
di marketplace*: la categoria merceologica di AiShop e del token. L'immagine azzurra
conteneva
inoltre le silhouette dei personaggi dei film, cioe' opera derivata.

Il rischio non e' teorico ma strutturale: adottato come marchio dell'ecosistema,
un'obiezione
costringerebbe a rifare l'identita' di sei prodotti insieme, nel momento in cui hanno
valore.

Il marchio costruito. Un triangolo con una traversa forma la **A**; il punto sopra e' la
i. Insieme: **Ai**, il prefisso che tiene insieme tutti i prodotti. Il triangolo e' anche

l'architettura reale: base larga di moduli, vertice di governance.

Geometricamente distinto: nessun cerchio inscritto, nessun asse verticale, nessuna figura.

Verificato per rendering a 512, 128, 48, 32 e 16 pixel — la prima versione, con tre elementi interni, era illeggibile sotto i 48 e va scartata.

File	Uso
<code>AiWorld-icona.svg</code>	tutti i moduli, tratto avorio
<code>AiEvents-icona.svg</code>	AiEvents, vetro azzurro con bagliore
<code>*-512.png</code> , <code>*-192.png</code>	sfondo trasparente, verificato canale alfa

Nell'app. L'icona e' incorporata come data URI: nessun file esterno, il vincolo del file unico regge. Vale sia per la scheda del browser sia per l'icona sulla schermata home. Il marchio compare anche accanto al nome in intestazione, disegnato in linea e colorato con la variabile del tema, quindi si adatta da solo a un eventuale cambio di palette.

Informazioni generali su marchi e diritti, non consulenza legale.

0.20.0 — merge sulla base testata

Il file 0.20.0 ricevuto non si avviava: due gestori puntavano a elementi rimossi (`btnSearchReal` , `badgeWarn`) e il primo errore interrompeva il 39% del codice. Sintassi valida, quindi invisibile senza eseguirlo. Scartato come baseline, come deciso.

Merge eseguito partendo da v0.19.1, che era gia' nel container: nessuna funzione riscritta, solo aggiunte e spostamenti.

Regola architetturale ratificata. Un elemento assente non deve mai poter spegnere l'app.

`$()` restituisce ora un segnaposto inerte invece di `null` : chiamare `addEventListener` ,

leggere o scrivere `value` su un elemento che non esiste non lancia piu' nulla. Verificato con un elemento inesistente.

AiSetup. I campi di vincoli e obiettivi non sono stati duplicati ma **spostati** da AiBadge: AiBadge resta i fatti del CV, AiSetup diventa la configurazione. Un solo `id` per campo, verificato con un test che conta le occorrenze. Aggiunti la scelta del paese (Svizzera,

Italia,
entrambe) che entra nella ricerca, e la verifica del punto di partenza sulla mappa.

AiAgents. Solo rinomina della scheda.

Colloqui in Oggi. La data dell'evento viene registrata alla creazione: il brief mostra "domani alle 14:30" e un colloquio entro un giorno sale in cima alla coda.

Foto profilo: non implementata. Era nella 0.20.0 ma non ha un uso: nessun canale di candidatura la richiede, e occuperebbe memoria del browser già vicina ai limiti. Da rivedere se AiApply un giorno compilerà moduli che la chiedono.

Sul simbolo proposto una seconda volta

La variante con lo yin-yang resta lo stesso marchio registrato: cerchio dentro triangolo con
asse verticale. Il contenuto interno non cambia la registrazione, che copre la
composizione.

La valutazione della 0.19.1 resta valida e non è stata modificata.